DocuMind AI

AI-Powered Research Document Assistant

Overview

DocuMind AI is a Python-based research document assistant. Users can upload PDF files and ask questions about their contents in natural language. The application uses a Retrieval-Augmented Generation (RAG) workflow: it extracts and processes document text, retrieves relevant passages through semantic search, and supplies those passages to an LLM to generate context-aware answers.

Key Features

- Upload PDF documents and extract text with PyMuPDF.
- Clean and split extracted text into smaller chunks for retrieval.
- Generate semantic embeddings with Sentence Transformers.
- Use FAISS for similarity-based retrieval of relevant document chunks.
- Use the Google Gemini API to generate answers grounded in retrieved context.
- Provide a FastAPI backend with document-upload and question-answering endpoints.
- Provide a Streamlit interface for uploading documents and asking questions.
- Return relevant source information, including document and page references, where available.

Technology Stack

Language: Python
PDF processing: PyMuPDF
Embeddings: Sentence Transformers
Vector search: FAISS
LLM: Google Gemini API
Backend: FastAPI
Frontend: Streamlit
API exploration: Swagger UI

How the RAG Workflow Works

1. PDF upload: the user provides a research document.
2. Extraction and preparation: PyMuPDF extracts text, which is cleaned and divided into chunks.
3. Embedding: Sentence Transformers converts each chunk into a numerical vector.
4. Retrieval: FAISS finds chunks whose vectors are most relevant to the user's question.
5. Generation: retrieved context is sent with the question to Gemini to generate an answer.
6. Sources: the application presents source information associated with retrieved passages.

Run Locally

1. Clone the repository and open its project directory.

```
git clone <your-repository-url>
cd <your-project-folder>
```

2. Create and activate a virtual environment on Windows.

```
python -m venv venv
venv\Scripts\activate
```

3. Install dependencies.

```
pip install -r requirements.txt
```

4. Configure your Gemini API key using the environment-variable name expected by your code. Example, if your application reads GEMINI_API_KEY:

```
set GEMINI_API_KEY=your_api_key_here
```

5. Start the backend and frontend in separate terminals. Adjust the FastAPI module path if your project uses a different import path.

```
uvicorn app.main:app --reload
streamlit run frontend.py
```

Project Status

The current milestone includes the PDF-processing and RAG workflow, FastAPI backend, and integrated Streamlit interface. SQLite-based document metadata, expanded automated tests, and systematic evaluation are planned improvements and should not be considered complete until implemented and verified.

Security

Do not commit API keys, passwords, virtual environments, uploaded private documents, or other secrets to GitHub. Keep credentials in environment variables or a local .env file, and ensure .env is listed in .gitignore.

Purpose

This project is a practical exploration of Generative AI and Retrieval-Augmented Generation, with the goal of making information retrieval from research documents more convenient and accessible.